

# Implementing and Testing a Probing-Based Path Verification for PolKA Source Routing Protocol

Henrique C. Layber, Roberta L. Gomes, Magnos Martinello

<sup>1</sup>Department of Informatics

Federal University of Espírito Santo (UFES) – Vitória, ES – Brazil

henrique.layber@edu.ufes.br, {roberta.gomes, magnos.martinello}@ufes.br

## **Abstract.**

**Resumo.** Este trabalho apresenta uma implementação prova-de-conceito e testes do **PathSec** [Martinello et al. 2024], um mecanismo de sondagem e verificação de rota para o protocolo de roteamento na origem PolKA. O mecanismo é baseado em uma combinação de funções hash, calculadas por switches de núcleo sem estado, e permitem uma parte confiável verificar se um pacote percorreu a rede ao longo do caminho definido pela origem. A implementação foi feita no Mininet usando BMv2 programados em P4.

**Keywords – Path Aware; Network Security; Path Verification; In-networking Programming**

## **1. Introduction**

Ever since *Source Routing* (SR) was proposed, there has been a need to ensure that packets traverse the network along the paths selected by the source, not only for security reasons, but also to ensure that the network is functioning properly and correctly configured. This is particularly important in the context of *Software-Defined Networks* (SDNs), where the control plane can select paths based on a variety of criteria[Jyothi et al. 2015].

In this paper, we propose an implementation on Programming Protocol Independent Processors (P4) for a Probing-Based Path Verification (PBV) mechanism originally proposed in **PathSec** [Martinello et al. 2024]. This mechanism provides proofs of packet forwarding, confirming that an expected route was indeed used for a packet. This is achieved by using a composition of hash functions on stateless core switches, combined with the switches native secret code (`node_ids`) and the respective route's output port, which results into a light-weight multi-signature. This signature can then be compared to a reference signature generated by a trusted party, the network controller.

All the code and test results are available online<sup>1</sup>.

## **2. Problem definition**

A **PathSec** network consists of a logically centralized SDN Controller that selects routing paths and configures the core and edge nodes. While ingress and egress edges encapsulates/decapsulates metadata into/from packets, core nodes execute basic packet forwarding with light signatures operations (proof-of-transit).

---

<sup>1</sup>[github.com/Henriquelay/polka-halftsiphash](https://github.com/Henriquelay/polka-halftsiphash)

**PathSec** is based on Polka’s source routing which explores the Chinese Remainder Theorem [Dominicini et al. 2020]. *PolKA* assigns a unique Route Identifier (`route_id`) to each path in the core network, and for each core switch it defines a secret `node_id`. Once a packet is associated to a specific route, each packet in this packet will carry in its header the respective `route_id` (embedded by the ingress edge). Then the core switches use the `route_id` and their respective `node_ids` to calculate the output `port_id` (it is a result of the Residue Number System (RNS)-based encoding [Martinello et al. 2024], which is out of the scope of this paper), routing the packets to the next core node in the expected path for the `route_id`.

Furthermore, **PathSec** designs a simplified signing mechanism using the uniqueness of the switch-port pair (`node_id`, `port_id`) for each `route_id` and its inherent secret component in the routing system, combined with cryptographic hashing functions. Every switch-port pair acts as a native secret code, enabling a lightweight multi-signature scheme based on Hash-based Message Authentication Code (HMAC): before routing a packet, a core node replaces the lightweight signature from the previous core node (embedded in the packet header), with a new one, hashing the previous one with the pair (`node_id`, `port_id`). This mechanism ensures an inherent property of the routing system that facilitates efficient path verification.

In the complete solution proposed by **PathSec**, once packets reach the egress edges, the final lightweight multi-signatures are extracted from the packets and sent to smart contracts deployed in a blockchain. These signatures are then compared to reference signatures previously registered by a trusted party (the network controller), allowing to verify packets’ *proof-of-transit* and to register path compliance.

## 2.1. Problem formalization

Let  $i$  be the source node (ingress node) and  $e$  be the destination node (egress node). Let the path from  $i$  to  $e$   $P_{i \rightarrow e}$  be a sequence of nodes: the path is defined by  $P_{i \rightarrow e} = (i, s_1, s_2, \dots, s_{n-1}, s_n, e)$ ,  $s_n$  being the  $n$ -th core node in the path. In SR protocols, packets’ routes through the core network are defined in  $i$ . That is,  $i$  sets the packet header with enough information for each core node to calculate the next hop.

The main problem we are trying to solve is path verification, that is, to have a way to ensure if the packet follows the path defined. A solution must be able to identify if the packet: (i) has passed through all nodes in the Path; (ii) has passed through the correct order of nodes; (iii) has **not** passed through any node that is **not** in the path.

More formally, given a routed sequence of nodes  $P_{i \rightarrow e}$ , and a captured sequence of nodes actually traversed  $P_j$ , a solution must identify if  $P_{i \rightarrow e} = P_j$ . Notably, it does not require validation, that is, it needs only to respond if  $P_{i \rightarrow e} = P_j$  and does not need to know any of their contents or check their validity as a route.

## 2.2. Multi-signature model for Path Verification

To balance trace effectiveness and scalability, **PathSec** proposes using packets themselves as probes [Martinello et al. 2024] [Ben Basat et al. 2020], with a lightweight multi-signature, keeping the header size fixed. That is, each core node hashes the previous node’s signature with a key only itself (and the Controller) can possess, and pass the hash

result forward, replacing the previous, and all following core nodes will do the same until the edge node is reached.

Each node’s execution plan is stateless. So, in terms of signature composition, a node  $n_i$  can be viewed as a function  $f_{n_i}(x)$ . A node can alter the header of the packet, which is used to push information downstream, allowing ultimately to detect if the path taken is correct.

In order to represent all nodes by the same function, we assign a distinct key value  $k$  for each node  $n_i$ , and use a bivariate function  $f(k_{n_i}, x) = f_{n_i}(x)$ . By using functions in two variables and enforcing one of the variables to have any value that’s unique between nodes, we ensure that the function’s result is unique for each switch as long as the function is collision resistant enough, that is,  $f_{n_a}(x) \neq f_{n_b}(x) \iff y \neq z$ .

Using function composition is appropriate, as it preserves the order-sensitive property of the path, since  $f \circ g \neq g \circ f$  in a general case. In our model, each node will execute a single function of this composition, using the previous node’s output as input. In this way,  $(f_{n_1} \circ f_{n_2} \circ f_{n_3})(x) = f(k_{n_3}, f(k_{n_2}, f(k_{n_1}, x)))$ ,  $f$  being a collision-resistant function, preferably a cryptographic hash function.

### 3. Implementing and Testing a Probing-Based Path Verification (PBV)

PBV consists of sending special packets (probes) with path-verification capabilities. The path verification system used is describe in subsection 2.2. Some assumptions are made: (i) Each node is assumed to be secure, that is, no node will alter the packet in any way that is not expected. This is a common assumption in SDN networks, where a trusted party is the only entity that can alter the network state; (ii) Every link is assumed to be perfect, that is, no packet loss, no packet corruption, and no packet duplication; (iii) Protocol boundary is IPv4, this means that **PathSec** is only used inside this network, and only IPv4 is used outside; (iv) All paths are assumed to be valid and all information correct unless stated otherwise; (v) Hash collisions will not happen. Furthermore, this implementation is a proof-of-concept, and there is little concerns about performance and protocol implementation: only IPv4 is supported (see item (iii)). The following subsections detail the setup and implementation.

#### 3.1. Setup

*P4* is a language used to program the data plane of network devices [Bosshart et al. 2014]. Behavioral Model 2 (*BMv2*) is a software switch that supports the *P4* language [Consortium 2025]. Mininet is a network emulation tool that allows the creation of virtual networks with a controlled environment [Lantz et al. 2010]. Mininet-wifi<sup>2</sup> is a Mininet fork used in this work due to its compatibility with *BMv2*. Despite the name, no Wi-Fi emulating feature was used.

The Controller is responsible for the network setup and for calculating reference signatures. In our implementation, the Python Script that controls Mininet acts a controller, and itself is not a network entity in Mininet. Scapy is a Python library that allows the creation and manipulation of packets [secdev 2003]. It is used to parse packets both for Controller operation and in the experiments and automate tests. The Tester is responsible for triggering Mininet hosts to send packets, setting up Scapy for parsing them and

---

<sup>2</sup>[github.com/intrig-unicamp/mininet-wifi](https://github.com/intrig-unicamp/mininet-wifi)

then asserting their signatures to Controller-calculated references. The setup is shown in Figure 1.

### 3.2. Implementation

This work extends previous work that implements *PolKA* in Mininet[Dominicini et al. 2021]<sup>3</sup>, by adding support to **PathSec**.

We define the header fields `timestamp` and `l_hash`, both 32-bits wide, to indicate the uniquely-generated number and the current light-weight multi-signature, respectively. Due to header incompatibility, probe packets operate using a different version number in the `version` field, so it is interoperable with *PolKA* by switching between probes and regular packets using the `version` header. *PolKA* packets use version `0x01`, and probe packets use version `0xF1`.

Figure 2 shows the used topology used in the experiments where  $s_i$  is a switch,  $e_i$  is an edge node, and  $h_i$  is a host. The topology is a linear chain with 10 switches and 11 hosts. The links are bidirectional, but only one direction is shown for simplicity.

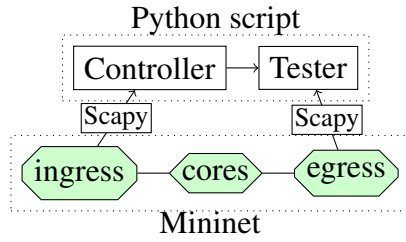


Figure 1: Experimentation setup

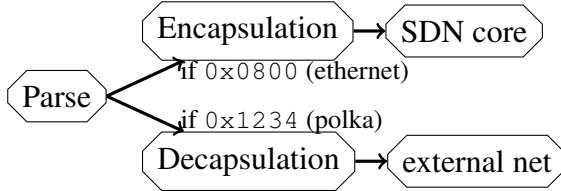


Figure 3: Edge ingress/egress steps

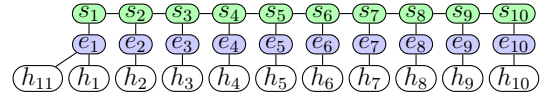


Figure 2: Baseline topology used in experimentation

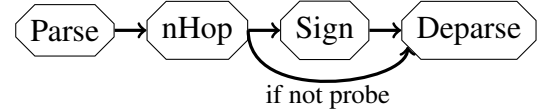


Figure 4: Data plane steps **Muito simples?**

Edge nodes can receive both *PolKA* packets or IPv4 packets. If the IPv4 protocol `ethertype` header field is detected (`0x0800`), it must be a packet from outside the network, so it proceeds with the role of an ingress edge and *PolKA* headers need to be added. Let this process be called *encapsulation*. If a *PolKA* protocol `ethertype` field is detected (`0x1234`), it must be a packet from the network core, so it proceeds with the role of an egress edge and the original IPv4 packet must be unwrapped. Let this process be called *decapsulation*. This flow is illustrated in Figure 3.

*PolKA* headers consists of the route polynomial (`route.id`), along with `version`, `ttl` and `proto` (stores the original `ethertype`). During encapsulation, the packet can be made into a probe packet by setting the `version` header field to `0xF1` and further defining the 32-bit fields `timestamp` to any unique value and `l_hash` initially equal to `timestamp`.

<sup>3</sup><https://github.com/nerds-ufes/polka>

On the core network, the hash function used is *SipHash-2-4-32*[Aumasson and Bernstein 2012] (*HalfSipHash*). After calculating the next hop (*nhop*) and if the packet is a probe packet *l\_hash* is computed as

$$l\_hash \leftarrow SipHash2-4-32(node\_id || nhop || timestamp || 0000000, l\_hash)$$

Meaning that a 64-bit string made of concatenating *node\_id*, *nhop*, *timestamp* with the remaining bits set to 0 for padding is used as key for the *HalfSipHash* function hashing *l\_hash*. This is illustrated on Figure 4.

## 4. Testing

This section presents the results for various tests representing different adversity scenarios. Each scenario is designed to evaluate the robustness and security of the system under specific attack or misconfiguration conditions.

### 4.1. Adversity scenarios

The scenarios include addition, detour, skipping, and out-of-order packet delivery. We define any switch that is not in the position defined in Figure 2 as an *attacker*, not only an intruder device, so it includes misconfigured switches, controller malfunction, and any other kind of network-affecting anomaly that results in these specified scenarios:

- i. **Addition** ( Figure 5.a ): An attacker switch is added between  $s_5$  and  $s_6$ . It perfectly replicates the behavior of the original switch, but it does not have the same *node\_id*, as it is a secret key, and the hashes will not match when validating the path past the attacker.
- ii. **Skipping** ( Figure 5.b ): A switch is removed between  $s_4$  and  $s_6$ . The packet will skip removed switch, and the hashes will not match when validating the path past the removed switch.
- iii. **Detour** ( Figure 5.c ): A switch is detoured between  $s_5$  and  $s_7$  into an attacker by hijacking the port that the packet should have taken exiting  $s_5$  and delivering the packet in a new port to  $s_7$ , posing as  $s_6$ . The hashes will not match when validating the path past the attacker.
- iv. **Out-of-order** ( Figure 5.c ): Links are changed between  $s_5$  and  $s_6$ . The packet will be delivered to  $s_6$  before  $s_5$ , and the hashes will not match when validating the path past the attack.

Table 1 shows the intermediary checksums for each scenario, highlighting the differences caused by the adversarial conditions, although in deployment, only the last *l\_hash* is every used for verification. Note that a scenario may result in a different amount of hashes to compare due to different amount of intermediary hops, so the table isn't completely filled to represent a hop that does not exist on a particular route. For all scenarios, a ping from  $h_1$  to  $h_{10}$  was performed, resulting in *route\_id* = 0x707b3a1d61d1d0d8b9fc91e442d0360dfc8bba4.

### 4.2. Discussion

Recovering data from Mininet switches is not trivial, as it does not provide a straightforward way to extract data from the switches. This limitation makes it difficult to analyze

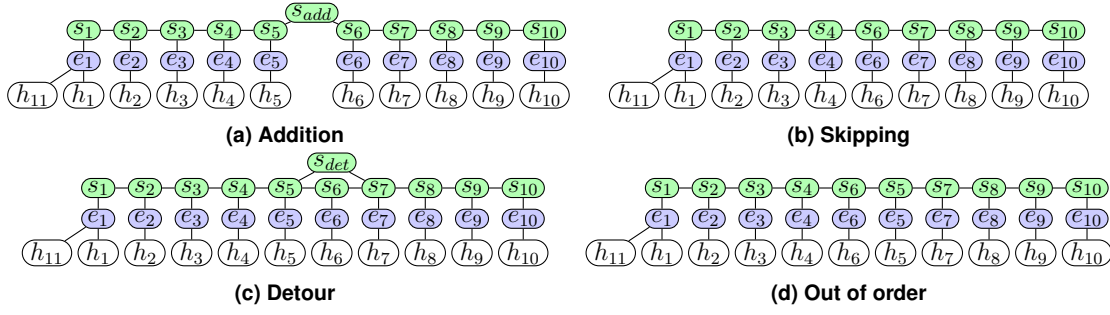


Figure 5: Topologies for tested scenarios

| Reference           | Addition             | Skipping            | Detour               | Out-of-order       |
|---------------------|----------------------|---------------------|----------------------|--------------------|
| $s_0$ 0x61e8d6e7    | $s_0$ 0x61e8d6e7     | $s_0$ 0x61e8d6e7    | $s_0$ 0x61e8d6e7     | $s_0$ 0x61e8d6e7   |
| $s_1$ 0xd25dc935    | $s_1$ 0xd25dc935     | $s_1$ 0xd25dc935    | $s_1$ 0xd25dc935     | $s_1$ 0xd25dc935   |
| $s_2$ 0x245b7ac5    | $s_2$ 0x245b7ac5     | $s_2$ 0x245b7ac5    | $s_2$ 0x245b7ac5     | $s_2$ 0x245b7ac5   |
| $s_3$ 0xa3b38b83    | $s_3$ 0xa3b38b83     | $s_3$ 0xa3b38b83    | $s_3$ 0xa3b38b83     | $s_3$ 0xa3b38b83   |
| $s_4$ 0x26aee736    | $s_4$ 0x26aee736     | $s_4$ 0x26aee736    | $s_4$ 0x26aee736     | $s_4$ 0x26aee736   |
| $s_5$ 0xf9b47914    | $s_5$ 0xf9b47914     |                     | $s_5$ 0xf9b47914     |                    |
|                     | $s_{add}$ 0x18c6d8d1 |                     |                      | $s_6$ 0x4b5a6c5a   |
| $s_6$ 0x18c6d8d1    | $s_6$ 0xb69b99ec     | $s_6$ 0x4b5a6c5a    | $s_{det}$ 0x250822a2 | $s_5$ 0xde3862a0   |
| $s_7$ 0xb69b99ec    | $s_7$ 0xfe6117f8     | $s_7$ 0x002346d3    | $s_7$ 0x40298bb9     | $s_7$ 0x648556ec   |
| $s_8$ 0xfe6117f8    | $s_8$ 0xc8d9fbde     | $s_8$ 0x7ec711aa    | $s_8$ 0xe13dcc9b     | $s_8$ 0x144e1d1b   |
| $s_9$ 0xc8d9fbde    | $s_9$ 0xa6293a25     | $s_9$ 0x5ee32b7b    | $s_9$ 0x1bf62c19     | $s_9$ 0x9e818f34   |
| $s_{10}$ 0xa6293a25 | $s_{10}$ 0xf4bcd07   | $s_{10}$ 0xc973a219 | $s_{10}$ 0xdd3a6675  | $s_{10}$ 0x6f694bc |

Table 1: Intermediary 1 hash for different test scenarios with the same seed

the data in real-time, so our solution depends on sniffer libraries (Scapy) to capture packets and extract the metadata. This limitation could be overcome by having the switches natively output the metadata to our controller, which would allow for a more performant and scalable solution. It was not implemented to keep the implementation minimal.

The cryptographically secure hash solution is only secure when the key is a secret. The `node_id` secret is output in `ttl PolKA` header field for debugging and presentation purposes, and this implementation should never be deployed in a real-world scenario. If the key is compromised, the entire system is compromised, as a malicious actor can easily generate the same checksums and be undetected, essentially signing whichever packets they want. Having the switch entry port included in the verification would mitigate this issue, as the attacker would need to know the exact port the packet entered the switch.

## 5. Conclusion and Future Work

As the examples show, our solution is able to detect when a packet is not following the expected path for most cases, and can be used to detect misconfigurations in the network.

As next steps, more parts of **PathSec** [Martinello et al. 2024] could be easily integrated for proof-of-concept, as all necessary information is already readily collect to Python scripts, making it simple to create a complete **PathSec** proof-of-concept software.

To improve the system for a real-world scenario, the ingress edge needs to relay the ingressed package metadata directly to the controller, and the egress edge will report the final checksum directly to the blockchain. Having it not passing through the controller.

Replay attacks are not detectable by the current solution, as the switch entry port is not included in the verification. This allows an attacker to replay a packet to a different

port, and the system will accept it as a valid packet. Including the entry port in the checksum would also be an appreciable increase in security, since it increases the number of targets an attacker would need to breach at the same time to be able to alter the path. It could be included in the *HalfSipHash* key, but a design decision is necessary: there are only 7 bits available, and the port number is 9 bits long. A possible solution would be to use the 8 least significant bits of the port number for both entry and exit port, limiting the number of ports checked before collision to 256, but this would be a reasonable trade-off for the increased security. So for a proof-of-concept, design was kept closed to the original **PathSec** description.

An interesting work can be done to use a rotating key architecture to detect replay attacks. This is a difficult problem to solve, since the key must be rotated and shared in a way that does not break the network, and of course, a malicious actor cannot acquire it. The key must be shared between the nodes in a secure, atomic way to prevent the network to enter an irrecoverable state. Pareço estar me estendendo mto, mas ainda acho um problema interessante e queria anotar isso de alguma coisa. Mas é um problema buscando uma solução. Manter de alguma forma?

Using a keyless hash function, or even a keyed one with hardcoded keys on deployment could be studied, as it would remove the need for a seed timestamp, but it would require to be a cryptographic hash, otherwise the data used to hash (which needs to include at least the `node_id`) could be derived from a switch's output. This would open ways to use a faster cryptographic hash function, such as xxHash's XXH3.

A timing analysis and stress tests can both be done to check if the incurred overhead is acceptable for the network, but this would require it to be implemented in hardware, not only in Mininet. The overhead needs to be low to represent a viable solution, otherwise the network will drop packets. Some overhead is acceptable since a configurable ratio/schedule of *PolKA* packets are selected to be probe packets.

Acabou meio do nadão, não? Agora bateu um desespero

## References

- [Aumasson and Bernstein 2012] Aumasson, J.-P. and Bernstein, D. J. (2012). SipHash: a fast short-input PRF. Cryptology ePrint Archive, Paper 2012/351.
- [Ben Basat et al. 2020] Ben Basat, R., Ramanathan, S., Li, Y., Antichi, G., Yu, M., and Mitzenmacher, M. (2020). Pint: Probabilistic in-band network telemetry. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '20*, page 662–680, New York, NY, USA. Association for Computing Machinery.
- [Bosshart et al. 2014] Bosshart, P., Daly, D., Gibb, G., Izzard, M., McKeown, N., Rexford, J., Schlesinger, C., Talayco, D., Vahdat, A., Varghese, G., and Walker, D. (2014). P4: programming protocol-independent packet processors. *SIGCOMM Comput. Commun. Rev.*, 44(3):87–95.
- [Consortium 2025] Consortium, P. L. (2025). Behavioral model. Accessed: 2025-03-15.
- [Dominicini et al. 2021] Dominicini, C., Guimarães, R., Mafioletti, D., Martinello, M., Ribeiro, M. R. N., Villaça, R., Loui, F., Ortiz, J., Slyne, F., Ruffini, M., and Kenny,

- E. (2021). Deploying polka source routing in p4 switches : (invited paper). In *2021 International Conference on Optical Network Design and Modeling (ONDM)*, pages 1–3.
- [Dominicini et al. 2020] Dominicini, C., Mafioletti, D., Locateli, A. C., Villaca, R., Martinello, M., Ribeiro, M., and Gorodnik, A. (2020). Polka: Polynomial key-based architecture for source routing in network fabrics. In *2020 6th IEEE Conference on Network Softwarization (NetSoft)*, pages 326–334.
- [Jyothi et al. 2015] Jyothi, S. A., Dong, M., and Godfrey, P. B. (2015). Towards a flexible data center fabric with source routing. In *Proceedings of the 1st ACM SIGCOMM Symposium on Software Defined Networking Research, SOSR '15*, New York, NY, USA. Association for Computing Machinery.
- [Lantz et al. 2010] Lantz, B., Heller, B., and McKeown, N. (2010). A network in a laptop: rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks, Hotnets-IX*, New York, NY, USA. Association for Computing Machinery.
- [Martinello et al. 2024] Martinello, M., Gomes, R. L., Borges, E. S., Layber, H. C., Bonella, V. B., Dominicini, C. K., Guimarães, R., Ribeiro, M., and Barcellos, M. (2024). Path-sec: Path-aware secure routing with native path verification and auditability. *Article*.
- [secdev 2003] secdev, B. P. (2003). Scapy. Accessed: 2024-09-26.